📋 Guía de Registro: Etapa 2

Microservicios Derivados - Paso a Paso

¿Qué es Etapa 2?

En la Etapa 2, implementarán un **servicio derivado** que:

- Usa datos de su propia colección (Etapa 1)
- Consume un microservicio de otro grupo via HTTP
- Produce información nueva combinando ambas fuentes

> **Ejemplo:** Si hicieron MS-clientes, ahora hacen clientes-resumen que consume MS-ordenes para calcular total gastado, cantidad de órdenes, etc.

Paso 1: Seleccionar su Servicio

Su grupo debe implementar **uno de estos servicios derivados**:

clientes-resumen	ordenes-detalle
productos-ficha	categorias-catalogo
inventario-alertas	sucursales-stock
ranking-productos	pagos-conciliacion
envios-demorados	resenas-criticas
promociones-precios	tickets-contexto

> ⚠️ **Importante:** El servicio depende de cuál fue su Etapa 1. Revisen la tabla en la guía oficial CONTRATO_ETAPA_2.md

---

Paso 2: Desarrollo Local

1 Crear estructura del proyecto

Crear un nuevo repositorio con esta estructura:

```
mi-servicio-etapa2/ ├── server.js ├── package.json ├── .env (local, NO commitear) ├── .env.example ├── README.md
└── test/ └── service.test.js
```

2 Implementar GET /health

Este endpoint es **obligatorio** y debe responder:

```
GET /health Status: 200 { "status": "ok", "timestamp": "2026-10-13T14:30:00.000Z", "service": "clientes-resumen" }
```

3 Implementar 2 endpoints derivados

Implementar según su servicio. Ejemplo para clientes-resumen:

```
GET /clientes/{id}/resumen GET /clientes/top?limit=5
```

Consultar **CONTRATO_ETAPA_2.md** para especificación exacta de respuestas JSON

4 Configurar variables de entorno

Crear archivo `.env` con:

```
PORT=3000 # URL del servicio que consumen (se obtiene del Dashboard) MS_ORDENES_URL=https://servicio-
upstream.onrender.com
```

> ⚠️ **Nunca commitear .env**
> Crear .env.example sin valores secretos para documentar variables

5 Probar localmente

```
npm install npm start # Verificar en otra terminal: curl http://localhost:3000/health
```

Paso 3: Desplegar en Render

1 Push a GitHub

```
git add . git commit -m "Implementar servicio clientes-resumen" git push origin main
```

2 Crear servicio en Render

1. Ir a **https://dashboard.render.com**
2. Clic en **New +**
3. Seleccionar **Web Service**
4. Conectar repositorio GitHub
5. Rellenar:
   - **Nombre:** nombre-servicio-etapa2
   - **Runtime:** Node
   - **Build Command:** npm ci
   - **Start Command:** npm start

3 Configurar variables de entorno

En Render, ir a **Environment** y agregar:

```
PORT = 3000 MS_ORDENES_URL = https://servicio-grupo-upstream.onrender.com
```

💡 Obtener la URL del servicio upstream del Dashboard en la página de Monitor (/monitor.html)

4 Esperar a que termine el deploy

Render construirá automáticamente. Esperar a que aparezca:

✓ Your service is live at: https://grupo-clientes-resumen.onrender.com

Pueden verificar visitando:

```
https://grupo-clientes-resumen.onrender.com/health
```

Paso 4: Obtener URL del Servicio Upstream

1 Abrir el Monitor del Dashboard

Visitar:

```
http://localhost:3000/monitor.html
```

(o la URL de producción si el Dashboard está deployado)

2 Buscar servicio upstream ONLINE

En el Monitor:

- Usar filtro **Etapa 1**
- Ver servicios con estado **✓ ONLINE** (verde)
- Ejemplo: Buscar "MS-ordenes" si necesitan consumir órdenes

```
┌─ MS-ordenes ✓ ONLINE │ Grupo: Equipo B │ URL: https://grupo-ordenes.onrender.com │ Endpoints: │ • GET /items │ • GET /items/{id} └─ Última verificación: OK (145ms)
```

3 Copiar la URL

Copiar la URL del servicio que consumen. Ejemplo:

```
https://grupo-ordenes.onrender.com
```

Esta URL va en la variable de entorno `MS_ORDENES_URL`

Paso 5: Registrar en el Dashboard

1 Ir a la página Home del Dashboard

```
http://localhost:3000
```

2 Hacer clic en "Registrar microservicio"

Se abrirá un diálogo con un formulario

3 Rellenar el formulario

Campo	Qué ingresar	Ejemplo
Microservicio	Seleccionar de la lista	clientes-resumen
URL base pública	URL de Render de su servicio	https://grupo-clientes-resumen.onrender.com
Nombre del grupo	Opcional, para identificación	Equipo A
Estudiantes	Uno por línea	Juan Pérez María García Pedro López

4 Hacer clic en "Registrar"

El Dashboard validará automáticamente:

- ✓ La URL es válida (http/https, pública)
- ✓ El servicio responde
- ✓ GET /health devuelve { status: "ok", ... }

Si todo es correcto:
"¡clientes-resumen registrado!"

Si algo falla:
Se mostrará un error específico. Revisar la URL y GET /health

Paso 6: Verificar en el Monitor

1 Abrir el Monitor

```
http://localhost:3000/monitor.html
```

2 Filtrar por "Etapa 2"

Ver solo servicios derivados para verificar que el suyo aparece

3 Verificar estado

```
┌─ clientes-resumen ✓ ONLINE │ Etapa 2 │ Equipo A: Juan Pérez, María García, Pedro López │ URL: https://grupo-clientes-resumen.onrender.com │ │ Endpoints: │ • GET /clientes/{id}/resumen │ • GET /clientes/top │ │ Última verificación: OK (234ms)
└─
```

Si ven **✓ ONLINE** (verde): ¡Éxito!
Si ven **✗ OFFLINE** (rojo): Revisar el error mostrado

Solución de Problemas

❌ "La URL no es válida"

- Verificar que sea https://, no http://
- No debe tener rutas (ej: /api)
- No debe tener puertos (ej: :3000)
- Ejemplo correcto: https://grupo-clientes-resumen.onrender.com

❌ "No se pudo conectar (timeout)"

- El servicio aún está deployándose en Render (esperar 2-3 minutos)
- Probar directamente: https://su-url.onrender.com/health
- Si responde ahí pero falla en el dashboard, esperar unos segundos y reintentar

❌ "GET /health: se esperaba { status: 'ok', ... }"

- El endpoint /health no existe o devuelve formato incorrecto
- Revisar server.js e implementar GET /health correctamente
- Debe responder: { "status": "ok", "timestamp": "...", ... }
- Redeploy en Render después de corregir

❌ "No se pudo conectar al servicio upstream"

- La variable de entorno MS_ORDENES_URL (o similar) está mal
- O el servicio upstream está offline
- Verificar en Monitor que el servicio que consumen esté ✓ ONLINE
- Revisar que la URL sea exacta (sin /health al final)

✅ Checklist Final

- ☐ Repositorio creado con código del servicio derivado
- ☐ GET /health implementado y responde correctamente
- ☐ 2 endpoints derivados implementados según contrato
- ☐ .env.example documentado con variables necesarias
- ☐ README.md con instrucciones de setup
- ☐ Push a GitHub
- ☐ Servicio desplegado en Render
- ☐ Variables de entorno configuradas en Render
- ☐ URL pública de Render obtenida
- ☐ URL del servicio upstream confirmada en Monitor
- ☐ Servicio registrado en Dashboard
- ☐ Servicio aparece en Monitor con estado ✓ ONLINE

📚 Referencias

CONTRATO_ETAPA_2.md
Especificación completa de endpoints para cada servicio derivado

ETAPA2_SETUP.md
Guía de setup técnico con ejemplos de código

Render Dashboard
https://dashboard.render.com
